

BioNix_Wesley_ILP_Week_1 to 4

Tuesday, 29 July 2025 20:10

What is Nix and BioNix

- Nix is a general build engine most commonly used for building software today, but as we will see a bit later it can also execute computational biology workflows in a pure manner with a small library called BioNix.

Pipeline in BioNix

- **What is Pipeline in BioNix**
 - o In BioNix, the term pipeline refers to an automated and modular sequence of steps designed to process and analyze bioinformatics data — typically from next-generation sequencing (NGS) — in a reproducible, efficient, and scalable manner.
- **What does BioNix pipeline do?**
 - o Standardize data processing
 - o Ensure reproducibility
 - o Support reproducible research computing
- Booleans: true and false
- Strings: "this is a string", can be multiline:
"
this is a
multiline
string
"
- Numbers: 0, 1.234
- Lists: [0 1.234 "string"]
- Attribute sets: { a = 5; b = "something else"; }
- Comments: # this is a comment
- Functions: x: x + 1
- Variable binding: let x = 5; in x $\Rightarrow$ 5
- Function application: let f = x: x + 1; in f 5 $\Rightarrow$ 6
- File paths: /path/to/file

Some common operators:

- Boolean disjunctions and conjunctions: true || false $\Rightarrow$ true, true && false $\Rightarrow$ false
- Ordering: 3 < 3 $\Rightarrow$ false, 3 <= 3 $\Rightarrow$ true
- Conditionals: if 3 < 4 then "a" else "b" $\Rightarrow$ a
- Addition and subtraction: 3 + 4 $\Rightarrow$ 7, 3 - 4 $\Rightarrow$ -1
- Multiplication and division: 3 * 4 $\Rightarrow$ 12, 3.0 / 4 $\Rightarrow$ 0.75
- String concatenation: "hello " + "world" $\Rightarrow$ "hello world"
- String interpolation: "hello \${"world"}" $\Rightarrow$ "hello world", "1 + 2 = \${toString (1 + 2)}" $\Rightarrow$ "1 + 2 = 3"
- Attribute set unions: { a = 5; } // { b = 6; } $\Rightarrow$ { a = 5; b = 6; }
- Let bindings: let x = 1; y = 2; in x + y $\Rightarrow$ 3

Nix is:

- Purely functional
- Lazy?? (Lazy valuation,
 - o Eg: although have th pre-requirement, it gonna show or contribute to the process when u use it.
- Purpose-built

Language constructs

Primitives / literals

Nix has a handful of data types which can be represented literally in source code, similar to many other languages.

```
# numbers
42
1.72394

# strings & paths
"hello"
./some-file.json

# strings support interpolation
"Hello ${name}"

# multi-line strings (common prefix whitespace is dropped)
''
first line
second line
''

# lists (note: no commas!)
[ 1 2 3 ]

# attribute sets (field access with dot syntax)
{ a = 15; b = "something else"; }

# recursive attribute sets (fields can reference each other)
rec { a = 15; b = a * 2; }
```

Operators

Nix has several operators, most of which are unsurprising:

Syntax	Description
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Numerical operations
<code>+</code>	String concatenation
<code>++</code>	List concatenation
<code>==</code>	Equality
<code>></code> , <code>>=</code> , <code><</code> , <code><=</code>	Ordering comparators
<code>&&</code>	Logical AND
<code> </code>	Logical OR
<code>e1 -> e2</code>	Logical implication (i.e. <code>!e1 e2</code>)
<code>!</code>	Boolean negation
<code>set.attr</code>	Access attribute <code>attr</code> in attribute set <code>set</code>
<code>set ? attribute</code>	Test whether attribute set contains an attribute

left // right
Merge left & right attribute sets, with the right set taking precedence

// (merge) operator

The // -operator is used pervasively in Nix code. You should familiarise yourself with it, as it is likely also the least familiar one.

It merges the left and right attribute sets given to it:

```
{ a = 1; } // { b = 2; }

# yields { a = 1; b = 2; }
```

Values from the right side take precedence:

```
{ a = "left"; } // { a = "right"; }

# yields { a = "right"; }
```

The merge operator does *not* recursively merge attribute sets;

```
{ a = { b = 1; }; } // { a = { c = 2; }; }

# yields { a = { c = 2; }; }
```

(Merge operator????)

1. What did the last interns do well, and what could they have done differently and why?
2. What are you interested in. What is the best way to contribute to AIVE and improve your skills in <insert interest here> and why?
3. What is Nix and what problem does it solve?
 - Nix is a purely functional package manager that builds and manages software in a reproducible, declarative, and isolated way.
4. What is BioNix and what problem does it solve?
 - BioNix is a collection of bioinformatics pipelines and tools, built and maintained at WEHI, using Nix for reproducibility and portability.
5. What software is required for VirusBreakend that is not available in nixpkgs?
 - VirusBreakend depends on GRIDSS (Genome Rearrangement IDentification Software Suite).
 - However, GRIDSS is not packaged in nixpkgs, so BioNix maintains its own expression for it.

<https://www.gnu.org/software/hello/>

Learning the Nix

Setting the plan

To start learning R and begin contributing effectively to the BioNix project, here's a focused roadmap tailored for you as a Master of Biotechnology student working on a research internship:

◆ Step 1: Understand What BioNix Is

BioNix is a reproducible bioinformatics pipeline framework using Nix for software packaging and environment management. It supports reproducible and portable bioinformatics workflows.

Key concepts to understand:

- What is a pipeline (in bioinformatics)?
- What problems does Nix solve? (dependency hell, reproducibility, portability)

👉 If you're new to Nix, I'll give you a mini crash course below.

◆ Step 2: Start Learning R for Bioinformatics

Since BioNix may involve working with R-based tools or data analysis, here's how to get up to speed:

📖 Basic R Learning (Week 1)

- Do: [R for Data Science](#) (free online book)
 - Focus on: tidyverse, ggplot2, dplyr, readr
- Practice: Use RStudio (or VSCode with R extension)
- Data: Start with public datasets like iris, mtcars, or any genomics CSV file

🔧 Bioinformatics-specific R (Week 2–3)

- Install and explore Bioconductor:

```
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install("GenomicRanges")
```

- Learn to:
 - Read FASTA/VCF files
 - Do basic sequence filtering or gene annotation
 - Visualize gene expression (e.g. with DESeq2, pheatmap)

◆ Step 3: Get Hands-on with BioNix (in parallel)

🧰 Tools You'll Need

- Nix: Learn how to use the Nix package manager
 - Start with: <https://nixos.org/learn.html>
 - Try basic commands: nix-shell, nix-env, nix-build

🔧 Try Running a Simple BioNix Pipeline

- Clone the BioNix repo you're working on
- Read the README.md and try to reproduce one of the example runs
- Understand the default.nix, shell.nix, and pipeline scripts (probably bash/snakemake)

📄 Read and Document

- Understand:
 - What inputs (data types, formats) the pipeline accepts
 - What tools it calls (e.g. samtools, bcftools, R scripts)
 - What outputs are generated

◆ Step 4: Integrate R with the Pipeline

- Write your own R script to:
 - Parse a .vcf file
 - Analyze read counts or mutation types
 - Visualize outputs (e.g. mutation spectrum)

Then ask your mentor how to hook it into the Nix-based pipeline (as a module or post-processing step).

◆ Step 5: Weekly Plan (Suggested)

Week	Focus
1	Learn R basics, install nix, try nix-shell
2	Try small R genomics tasks (Bioconductor), run existing pipeline
3	Understand pipeline structure, add your own R analysis
4	Document what you've done, ask for feedback, iterate

1. AD hoc shell environment
<https://nix.dev/tutorials/first-steps/ad-hoc-shell-environments>

1.1 Create a shell environment

Q: trouble to downloading the Nix macOS

Updating to macOS 15 Sequoia

If you recently updated to macOS 15 Sequoia and are getting error: the user '_nixbld1' in the group 'nixbld' does not exist when running Nix commands, refer to GitHub issue [NixOS/nix#10892](#) for instructions to fix your installation without reinstalling.

Which still getting trouble for downloading, the computer environment is not allowed

Once showing and running two exotic programs called <cowsay> and <lolcat> that not have installed on

Use nix-shell with the -p (--packages) option to specify that we need the cowsay and lolcat packages:

Note:

```
$ nix-shell -p cowsay lolcat
these 3 derivations will be built:
/nix/store/zz1j8gchgwzfn7sr4r8yxb7a0afkjdg-builder.pl.drv
/nix/store/h9sbaa2k8ivnihw2czhl5b58k0f7fsfh-lolcat-100.0.1.drv
...
[nix-shell:~]$
```

Within the Nix shell, you can use the programs provided by these packages:

Note:

```
[nix-shell:~]$ cowsay Hello, Nix! | lolcat
```

Exit the shell: type <exit> or press<CTRL-D>

```
[nix-shell:~]$ exit
exit
$ cowsay no more
The program 'cowsay' is currently not installed.
```

```
$ echo all gone | lolcat
```

The program 'lolcat' is currently not installed.

[\[nix-shell:~} like to creat a nix shell environment](#)

\$ symbol like the `` in R code

1.2 Running programs once

If running any program directly

Eg:

```
$ nix-shell -p cowsay --run "cowsay Nix"
```

If only program name

Eg:

```
$ nix-shell -p hello --run hello
```

No quotes are needed

1.3 Search for package

Tip: Enter the program name you want to run in search.nixos.org to find packages that provide it.

!!! A package name is not the same as a program name. Many packages provide multiple programs, or no programs at all if they are libraries. Even for packages that provide exactly one program, the package and program name are not necessarily the same.

1.4 Run any combination of programs

The -p (--packages) argument can take multiple package names. After you have the package name and shell with the package.

Start a Nix shell with the packages providing git, nvim, and npm.

1.5 check package versions

1.6 Nested shell sessions

If need an additional program temporarily, can run a nested Nix shell.

The programs provided by the specified packages will be added to the current environment.

!!! Exit the shell as usual to return to the previous environment.

Q:

1. Why might nix shell be preferred for testing tools in CI pipelines or temporary debugging sessions?

nix shell is a command that launches a temporary environment with specified packages available, without installing them globally. It's useful for testing tools, running scripts, or debugging in a clean, reproducible setup.

A CI pipeline (Continuous Integration pipeline) is an automated process that runs every time code is pushed to a repository. It typically includes:

- Code checkout – pull latest changes from the repo
- Build – compile code or set up the environment
- Test – run unit/integration tests
- Lint – check code style or formatting
- Package/Deploy – optionally create artifacts or deploy to a test/staging environment

Purpose:

To catch bugs early, ensure code quality, and automate repetitive tasks like testing and deployment.

Tools: GitHub Actions, GitLab CI, Jenkins, CircleCI, etc.

Why debugging session?

A debugging session is a temporary environment where developers investigate and fix issues in software.

Because Nix gives you a clean, reproducible, and isolated environment that's perfect for debugging software issues.

1. No system pollution
Use nix shell to add only the tools you need (e.g., gdb, strace, jq) — nothing is permanently installed.
2. Reproducibility
The exact same debug environment can be recreated on any machine or CI runner using a single command.
3. Version control of tools
Easily test bugs with different versions of a package:
`nix shell nixpkgs#python38 (advantages?)`
4. Dependency isolation
Prevents conflicts from system-wide libraries or tools that might interfere with your debugging.
5. Safe experimentation
You can try different configurations, compilers, or flags in temporary shells without breaking anything.

```
nix shell nixpkgs#python38 (advantages?)
```

There are three things going on here:

1. `--run` executes the given [Bash command](#) within the environment created by Nix, and exits when it's done. You can use this with `nix-shell` whenever you want to quickly run a program you don't have installed on your machine.
2. `--pure` discards most environment variables set on your system when running the shell. It means that only the git provided by Nix is available inside that shell. This is useful for simple one-liners such as in the example. While developing, however, you will usually want to have your editor and other tools around. Therefore we recommend to omit `--pure` for development environments, and to add it only when the extra isolation is needed.
3. `-I` determines what to use as a source of package declarations. Here we provided [a specific Git revision of nixpkgs](#), leaving no doubt about which version of the packages in that collection will be used.

Week 4

- Based on the tutorial to practice Nix
- First tutorial

- Create a shell environment

- Test the Nix Version

```
MacBookPro:~ wesley$ nix --version
nix (Nix) 2.30.2
```

- Use `nix-shell` with the `-p` (`--packages`) option to specify that we need the `cowsay` and `lolcat` packages:

```
MacBookPro:~ wesley$ nix-shell -p cowsay lolcat
[nix-shell:~]$
```

- Within the Nix shell, you can use the programs provided by these packages:

```
[nix-shell:~]$ nix-shell -p cowsay lolcat
```

```
[nix-shell:~]$ cowsay Hello, Nix! | lolcat
```

< Hello, Nix! >

$$\frac{\hat{(\overline{00})}}{\hat{()}} \frac{\hat{()}}{\hat{()}}$$

- Type exit or press CTRL-D to exit the shell, and the programs won't be available anymore.

```
[nix-shell:~]$
cowsay no more
```

```
echo all gone | lolcat
```

```
< no more >
-----
\      ^ ^
 \    (oo)\_____)
  (__) \       )\\/
       ||----w |
       ||     ||
```

```
all gone
```

```
[nix-shell:~]$
```

○ Running program once

- You can go even faster, by running any program directly:

```
[nix-shell:~]$ nix-shell -p cowsay --run "cowsay Nix"
```

```
< Nix >
-----
\      ^ ^
 \    (oo)\_____)
  (__) \       )\\/
       ||----w |
       ||     ||
```

- If the command consists only of the program name, no quotes are needed:

```
[nix-shell:~]$ nix-shell -p hello --run hello
Hello, world!
```

○ Search for a package

- **Tips!!!**

- Enter the program name you want to run in search.nixos.org to find packages that provide it.

For the following example, find the package names for these programs:

- git
- nvim
- npm

In the search results, each item shows the package name, and the details list the available programs.

○ Run any combination of program

- Once you have the package name, you can start a shell with that package. The -p(--packages) argument can take multiple package names.

Start a Nix shell with the packages providing git, nvim, and npm. Again, the first invocation may take a while to download all dependencies.

○ Check package version

```
[nix-shell:~]$ which git
/nix/store/3cdi52xh6lk3h1fb51jkxs3p561p37wg-git-2.38.3/bin/git
```

```
[nix-shell:~]$ git --version
git version 2.38.3
```

```
[nix-shell:~]$ which nvim
/nix/store/ynskzgkf07lmrrs3cl2kzr9ah487lwab-neovim-0.8.1/bin/nvim
```

```
[nix-shell:~]$ nvim --version | head -1
NVIM v0.8.1
```

```
[nix-shell:~]$ which npm
```



```
[nix-shell:~]$ which npm
/nix/store/q12w83z0i5pi1y0m6am7qmw1r73228sh-nodejs-18.12.1/bin/npm
```

```
[nix-shell:~]$ npm --version
8.19.2
```

E325: ATTENTION

```
Found a swap file by the name "~/.local/state/nvim/swap/%Users%wesley%
v0.8.1.swp"
```

```
    owned by: wesley    dated: Thu Aug 07 03:24:59 2025
    file name: ~wesley/v0.8.1
    modified: YES
    user name: wesley    host name: MacBookPro
    process ID: 1517
```

```
While opening file "v0.8.1"
CANNOT BE FOUND
```

```
(1) Another program may be editing the same file.  If this is the case,
    be careful not to end up with two different instances of the same
    file when making changes.  Quit, or continue with caution.
```

```
(2) An edit session for this file crashed.
    If this is the case, use ":recover" or "nvim -r v0.8.1"
    to recover the changes (see ":help recovery").
    If you did this already, delete the swap file
```

```
"/Users/wesley/.local/state/nvim/swap/%Users%wesl
ey%v0.8.1.swp"
```

```
to avoid this message.
```

```
Swap file "~/.local/state/nvim/swap/%Users%wesley%v0.8.1.swp" already exists!
[O]pen Read-Only, [E]dit anyway, [R]ecover, [D]elete it, [Q]uit, [A]bort:
```

```
?????
```

```
Type q
```

```
Then
```

```
[nix-shell:~]$ [nix-shell:~]$ which git
/nix/store/3cdi52xh6lk3h1fb51jkxs3p561p37wg-git-2.38.3/bin/git
```

```
[nix-shell:~]$ git --version
git version 2.38.3
```

```
[nix-shell:~]$ which nvim
/nix/store/ymskzgf07lmrrs3cl2kzr9ah487lwab-neovim-0.8.1/bin/nvim
```

```
[nix-shell:~]$ nvim --version | head -1
NVIM v0.8.1
```

```
[nix-shell:~]$ which npm
/nix/store/q12w83z0i5pi1y0m6am7qmw1r73228sh-nodejs-18.12.1/bin/npm
```

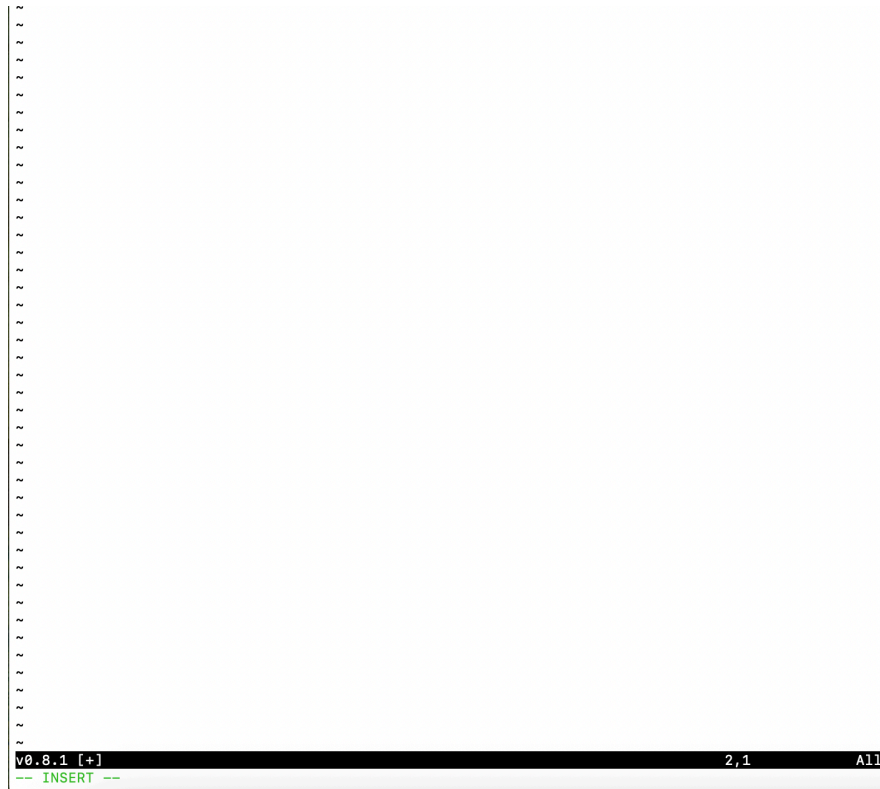
```
[nix-shell:~]$ npm --version
8.19.2
bash: [nix-shell:~]$: command not found
bash: /nix/store/3cdi52xh6lk3h1fb51jkxs3p561p37wg-git-2.38.3/bin/git: No such
file or directory
bash: [nix-shell:~]$: command not found
git version 2.50.1
bash: [nix-shell:~]$: command not found
bash: /nix/store/ymskzgf07lmrrs3cl2kzr9ah487lwab-neovim-0.8.1/bin/nvim: No
such file or directory
bash: [nix-shell:~]$: command not found
bash: [nix-shell:~]$: command not found
bash: /nix/store/q12w83z0i5pi1y0m6am7qmw1r73228sh-nodejs-18.12.1/bin/npm: No
such file or directory
bash: [nix-shell:~]$: command not found
bash: 8.19.2: command not found
```

The shell (command) not be found, should I install first?

Try again

Type d

```
exit
|
~
~
~
~
~
```



Can not be solved. Question: why the page like this??? What is that mean?

THIS ONE CANNOT RUN UNDER ANY SHELL ENVIRONMENT!!!

Ask chatgpt

From the screenshots, it looks like:

1. **Inside nix-shell** — You could run which git, which nvim, which npm and they resolved to Nix store paths, and running them worked fine.
2. **Outside nix-shell** — Those paths no longer existed in your environment, so git, nvim, and npm commands were not found. That's why you saw errors like:
bash: /nix/store/.../bin/git: No such file or directory

This happens because Nix packages are only available inside the nix-shell unless you install them globally or add them to your shell profile.

3. **The nvim swap file warning** — The E325: ATTENTION message just means a previous nvim session crashed or didn't close cleanly. You can:
 - o Recover: nvim -r filename
 - o Delete the swap: rm ~/.local/state/nvim/swap/%Users%wesley%v0.8.1.swp
 - o Or open normally by choosing (E) or (O) in the prompt.

How to fix the “command not found” issue

You have two main options:

- **Use nix-shell every time** you need these tools (re-enter the same nix-shell with your shell.nix or nix-shell -p git neovim nodejs).
- **Install them globally** with Nix:
nix-env -iA nixpkgs.git nixpkgs.neovim nixpkgs.nodejs

This makes them available outside of nix-shell.

If you want, I can give you the exact commands to make git, nvim, and npm available permanently so you don't have to enter nix-shell every time. Would you like me to prepare that?

prepare that:

- The solution GPT gives to me
Great—here are the clean, permanent installs (no nix-shell needed).

1) Install globally to your user profile

Pick ONE Node version (18 = same as your screenshot; 20 = current LTS)

nix profile install 'nixpkgs#git' 'nixpkgs#neovim' 'nixpkgs#nodejs_18'

or

nix profile install 'nixpkgs#git' 'nixpkgs#neovim' 'nixpkgs#nodejs_20'

2) Make sure your PATH includes your Nix profile (do this once)

Add this to your shell rc (e.g., ~/.zshrc or ~/.bashrc), then open a new terminal:

export PATH="\$HOME/.nix-profile/bin:/nix/var/nix/profiles/default/bin:\$PATH"

3) Verify

which git nvim npm

git --version

nvim --version | head -1

npm --version

Notes

- Don't copy the prompt text like [nix-shell:~]\$—that's why you saw command not found.
- The swap warning in Neovim is safe to clear:
rm ~/.local/state/nvim/swap/%Users%wesley%v0.8.1.swp

(Or press D at the prompt in nvim.)

If you prefer system-wide (all users), you can also use Home Manager or nix-darwin, but the above is the quickest.

- Run back to Nix environment
MacBookPro:~ wesley\$ # Pick ONE Node version (18 = same as your screenshot; 20 = current LTS)
MacBookPro:~ wesley\$ nix profile install 'nixpkgs#git' 'nixpkgs#neovim' 'nixpkgs#nodejs_18'
warning: 'install' is a deprecated alias for 'add'
error: experimental Nix feature 'nix-command' is disabled; add '--extra-experimental-features nix-command' to enable it
MacBookPro:~ wesley\$ # or
MacBookPro:~ wesley\$ # nix profile install 'nixpkgs#git' 'nixpkgs#neovim' 'nixpkgs#nodejs_20'
MacBookPro:~ wesley\$ export PATH="\$HOME/.nix-profile/bin:/nix/var/nix/profiles/default/bin:\$PATH"
MacBookPro:~ wesley\$ which git nvim npm
/usr/bin/git
MacBookPro:~ wesley\$ git --version
git version 2.39.5 (Apple Git-154)
MacBookPro:~ wesley\$ nvim --version | head -1
-bash: nvim: command not found
MacBookPro:~ wesley\$ npm --version
-bash: npm: command not found
MacBookPro:~ wesley\$ rm ~/.local/state/nvim/swap/%Users%wesley%v0.8.1.swp
MacBookPro:~ wesley\$

- I think the problem appear iin nvim/npm command, those are not be found, but I do follow the instruction to practice the code

o Nested shell session

- If you need an additional program temporarily, you can run a nested Nix shell. The programs provided by the specified packages will be added to the current environment.

[nix-shell:~]\$ nix-shell -p python3

this path will be fetched (11.42 MiB download, 62.64 MiB unpacked):

/nix/store/pwy30a7siqrkki9r7xd1lksyv9fg4l1r-python3-3.10.11

copying path '/nix/store/pwy30a7siqrkki9r7xd1lksyv9fg4l1r-python3-3.10.11' from '<https://cache.nixos.org/>'

[https://wiki.nixos.org/...](https://wiki.nixos.org/)

```
[nix-shell:~]$ python --version
Python 3.10.11
```

- Towards reproducibility

These shell environments are very convenient, but the examples so far are not reproducible yet. Running these commands on another machine may fetch different versions of packages, depending on when Nix was installed there.

What do we mean by reproducible? A fully reproducible example would give exactly the same results no matter when or where you run the command. The environment provided would be identical each time.

The following example creates a fully reproducible environment. You can run it anywhere, anytime to obtain the exact same version of the git.

- `nix-shell -p git --run "git --version" --pure -I nixpkgs=https://github.com/NixOS/nixpkgs/tarball/2a601aafdc5605a5133a2ca506a34a3a73377247`

```
[nix-shell:~]$ nix-shell -p git --run "git --version" --pure -I
nixpkgs=https://github.com/NixOS/nixpkgs/tarball/2a601aafdc5605a5133a2ca506a3
4a3a73377247
unpacking
'https://github.com/NixOS/nixpkgs/tarball/2a601aafdc5605a5133a2ca506a34a3a733
77247' into the Git cache...
git version 2.33.1
```

There are three things going on here:

1. `--run` executes the given [Bash command](#) within the environment created by Nix, and exits when it's done.
You can use this with `nix-shell` whenever you want to quickly run a program you don't have installed on your machine.
2. `--pure` discards most environment variables set on your system when running the shell. It means that only the git provided by Nix is available inside that shell. This is useful for simple one-liners such as in the example. While developing, however, you will usually want to have your editor and other tools around. Therefore we recommend to omit `--pure` for development environments, and to add it only when the extra isolation is needed.
3. `-I` determines what to use as a source of package declarations.
Here we provided [a specific Git revision of nixpkgs](#), leaving no doubt about which version of the packages in that collection will be used.

- Reproducibility interpreted script

- A trivial script with non-trivial dependencies
 - Take the following script, which fetches the content XML of a URL, converts it to JSON, and formats it for better readability:

```
[nix-shell:~]$ #! /bin/bash

curl https://github.com/NixOS/nixpkgs/releases.atom | xml2json | jq .
bash: xml2json: command not found
% Total    % Received % Xferd  Average
Speed    Time      Time      Time    Current
Dload    Upload   Total   Spent    Left     Speed
100 5544  100 5544    0     0  9307      0 --:--:-- --:--:-- --:--:-- 930
2
curl: Failed writing body
```

The error in `xml2json: command not found` & `curl: Failed writing body`

Try to fix it

1. The error in `xml2json: command not found`
 - Using `nix-shell -p python3Packages.xml2json`

```
[nix-shell:~]$ nix-shell -p python3Packages.xml2json
error:
... while calling the 'derivationStrict' builtin
at <nix/derivation-internal.nix>:37:12:
```

```

36|
37|     strict = derivationStrict drvAttrs;
   |           ^
38|

... while evaluating derivation 'shell'
   whose name attribute is located at
/nix/store/gp8yc3qdcgzw3b0p3gzlhiy57smq25k-
nixpkgs/nixpkgs/pkgs/stdenv/generic/make-derivation.nix:482:13

... while evaluating attribute '__impureHostDeps' of derivation
'shell'
   at /nix/store/gp8yc3qdcgzw3b0p3gzlhiy57smq25k-
nixpkgs/nixpkgs/pkgs/stdenv/generic/make-derivation.nix:641:15:
640|         );
641|         ^__impureHostDeps =
   |           ^
642|         computedImpureHostDeps

(stack trace truncated; use '--show-trace' to show the full,
detailed trace)

error: attribute 'xml2json' missing
at «string»:1:107:
   1| {...}@args: with import <nixpkgs> args;
     |                                     (pkgs.runCommandCC or pkgs.runCommand) "shell" { buildInputs =
     |                                     [ (python3Packages.xml2json) ]; } ""
     |                                     |
Did you mean xmljson?

```

- Using nix-shell -p nodePackages.xml2json this time

```

[nix-shell:~]$ nix-shell -p nodePackages.xml2json
error:
... while calling the 'derivationStrict' builtin
   at <nix/derivation-internal.nix>:37:12:
   36|
   37|     strict = derivationStrict drvAttrs;
     |           ^
   38|

... while evaluating derivation 'shell'
   whose name attribute is located at
/nix/store/gp8yc3qdcgzw3b0p3gzlhiy57smq25k-
nixpkgs/nixpkgs/pkgs/stdenv/generic/make-derivation.nix:482:13

... while evaluating attribute '__impureHostDeps' of derivation 'shell'
   at /nix/store/gp8yc3qdcgzw3b0p3gzlhiy57smq25k-
nixpkgs/nixpkgs/pkgs/stdenv/generic/make-derivation.nix:641:15:
640|         );
641|         ^__impureHostDeps =
   |           ^
642|         computedImpureHostDeps

(stack trace truncated; use '--show-trace' to show the full, detailed
trace)

error: attribute 'xml2json' missing
at «string»:1:107:
   1| {...}@args: with import <nixpkgs> args; (pkgs.runCommandCC or
     |                                     pkgs.runCommand) "shell" { buildInputs = [ (nodePackages.xml2json) ]; } ""
     |                                     |

```

???????????

Maybe no xml2json package in my nixpkgs set

Try to using "

Latest nixpkgs release tag

```

curl -s https://api.github.com/repos/NixOS/nixpkgs/releases/latest | jq -r '.tag_name'

```

Or list the most recent 5 tags

```

curl -s https://api.github.com/repos/NixOS/nixpkgs/tags | jq -r '.[0:4].name' | head -n 5

```

```

[nix-shell:~]$ curl -s
https://api.github.com/repos/NixOS/nixpkgs/releases/latest | jq -r
'.tag_name'
null

```

```
[nix-shell:~]$ curl -s https://api.github.com/repos/NixOS/nixpkgs/tags | jq -r '.[].name' | head -n 5
v208
v206
v192
release-16.03-start
last-glibc-2.13
```

Seems working

2. Back to curl: Failed writing body

```
- Using
curl -s https://github.com/NixOS/nixpkgs/releases.atom \
  | xml2json \
  | jq .
[nix-shell:~]$ curl -s https://github.com/NixOS/nixpkgs/releases.atom \
  | xml2json \
  | jq .
bash: xml2json: command not found
```

Still not working

Q: the error occur in xml2json can't be found?????
Not really sure about the newest pkgs include it or not

Cuz based on tutorial said "

It requires the programs curl, xml2json, and jq. It also requires the bash interpreter. If any of these dependencies are not present on the system running the script, it will fail partially or altogether.

With Nix, we can declare all dependencies explicitly, and produce a script that will always run on any machine that supports Nix and the required packages taken from Nixpkgs."

If the problem of xml2json can not be solve. Then the tutorial of Reproducibility interpreted script cannot be continue

Temporally skip...

○ Declarative shell environments with shell.nix

Entering a temporary shell

Suppose we want an environment where cowsay and lolcat are available. The simplest possible way to accomplish this is via the nix-shell -p command:

```
$ nix-shell -p cowsay lolcat
```

This command works, but there's a number of drawbacks:

- You have to type out -p cowsay lolcat every time you enter the shell.
- It doesn't (ergonomically) allow you any further customization of your shell environment.

A better solution is to create our shell environment from a shell.nix file.

○ A basic shell.nix file

Create a file called shell.nix with these contents:

```
let
  nixpkgs = fetchTarball "https://github.com/NixOS/nixpkgs/tarball/nixos-24.05";
  pkgs = import nixpkgs { config = {}; overlays = []; };
in
  pkgs.mkShellNoCC {
    packages = with pkgs; [
```

```

    cowsay
    lolcat
  ];
}

```

Run

```

[nix-shell:~]$ let
  nixpkgs = fetchTarball
  "https://github.com/NixOS/nixpkgs/tarball/nixos-24.05";
  pkgs = import nixpkgs { config = {}; overlays = []; };
in

pkgs.mkShellNoCC {
  packages = with pkgs; [
    cowsay
    lolcat
  ];
}
bash: let: expression expected
bash: nixpkgs: command not found
bash: syntax error near unexpected token `}'
bash: syntax error near unexpected token `in'
bash: pkgs.mkShellNoCC: command not found
bash: packages: command not found
bash: [: missing `]'

```

Seems ive got shell environment

No, seems this one have to under the `$ nix-shell -p cowsay lolcat` environment

```

[nix-shell:~]$ let
  nixpkgs = fetchTarball
  "https://github.com/NixOS/nixpkgs/tarball/nixos-24.05";
  pkgs = import nixpkgs { config = {}; overlays = []; };
in

pkgs.mkShellNoCC {
  packages = with pkgs; [
    cowsay
    lolcat
  ];
}
bash: let: expression expected
bash: nixpkgs: command not found
bash: syntax error near unexpected token `}'
bash: syntax error near unexpected token `in'
bash: pkgs.mkShellNoCC: command not found
bash: packages: command not found
bash: [: missing `]'

```

But whyyyyyyyyy?

Try to use gpt's solution

You pasted **Nix code into the Bash prompt**. Bash can't evaluate Nix, so it threw those "command not found / syntax error" messages. Do this instead:

Quick way (no file)

Spawns a shell that has cowsay and lolcat available

`nix-shell -p cowsay lolcat`

Proper way with shell.nix

1. Create a folder and the file:

```
mkdir -p ~/nix-demo && cd ~/nix-demo
```

```
cat > shell.nix <<'EOF'
```

```
{ pkgs ? import (fetchTarball
```

```
"https://github.com/NixOS/nixpkgs/archive/nixos-24.05.tar.gz") {} }:
```

```
pkgs.mkShell {
```

```
  packages = with pkgs; [
```

```
    cowsay
```

```
    lolcat
```

```
  ];
```

```
}
EOF
1. Enter the dev shell (this reads shell.nix):
nix-shell
1. Use the tools:
cowsay "hi nix" | lolcat
```

If you prefer your original snippet

Your snippet is fine as a file, not at the prompt. Save **exactly** this as shell.nix:

```
let
  nixpkgs = fetchTarball
    "https://github.com/NixOS/nixpkgs/archive/nixos-24.05.tar.gz";
  pkgs = import nixpkgs { config = {}; overlays = []; };
in
  pkgs.mkShellNoCC {
    packages = with pkgs; [
      cowsay
      lolcat
    ];
  }
}
```

Then run nix-shell in that directory.

Common gotchas

- Don't type let ... in ... into Bash; it belongs in shell.nix.
- Use the **archive** URL ending in .tar.gz (works better with fetchTarball).
- If you see error: experimental Nix feature 'nix-command' is disabled, add:


```
mkdir -p ~/.config/nix
printf "experimental-features = nix-command flakes\n" >> ~/.config/nix/nix.conf
```

(Restart your shell.)

I have to restart the shell, to re-try, why?????

Retry though Gpt solution

```
MacBookPro:~ wesley$ # Spawns a shell that has cowsay and lolcat available
MacBookPro:~ wesley$ nix-shell -p cowsay lolcat
```

```
[nix-shell:~]$
```

```
[nix-shell:~]$ mkdir -p ~/nix-demo && cd ~/nix-demo
```

```
cat > shell.nix <<'EOF'
{ pkgs ? import (fetchTarball
  "https://github.com/NixOS/nixpkgs/archive/nixos-24.05.tar.gz") {} }:
  pkgs.mkShell {
    packages = with pkgs; [
      cowsay
      lolcat
    ];
  }
}
EOF
```

```
[nix-shell:~/nix-demo]$ nix-shell
unpacking 'https://github.com/NixOS/nixpkgs/archive/nixos-24.05.tar.gz' into
the Git cache...
these 3 derivations will be built:
  /nix/store/91i2w23fvrzkxpjjbsp684h4r7rlr1-builder.pl.drv
  /nix/store/zig0y28vz6y2dx7i5vh0pmrj7plgvamq-lolcat-100.0.1.drv
  /nix/store/n76nv3d8zl36q4vhkvxmh056k7q5kb4i-lolcat-100.0.1.drv
these 74 paths will be fetched (162.13 MiB download, 1021.20 MiB unpacked):
  /nix/store/f9wk3hmrxc1wyh6vvsncbgg4f7wcsddg-Libsystem-1238.60.2
  /nix/store/jyplylm4rx3zfpw0nxvji5d991ax5gci-adv_cmds-119-locale
  /nix/store/9pzls8barp493anxxik2ngf8q9jag2sx-apple-framework-CoreFoundation
  /nix/store/7ia0cb68r08jc0zv437a7k9jsm42gcyr-bash-5.2p32
  /nix/store/lfb99d5radvax528l6aadvd5a8fa7lpm-binutils-2.41
```



```
/nix/store/dqwq90k5arnpsj030avqjlp8ikqyyjbw-binutils-2.41-lib
/nix/store/2y36i04pshmfyz4a6nd3jiihlngz4c8-bundler-2.5.9
/nix/store/n9cp4mmgya6pcglpk58bzhdrwq8618-bzip2-1.0.8
/nix/store/f3q95y31k4smw1qwcgvf0mpm4n9la7yn-bzip2-1.0.8-bin
/nix/store/268cqlnw6ww917p8alrlslwjxydyqvqc-cctools-binutils-
darwin-16.0.6-973.0.1
/nix/store/rgiirnhx8b9ign0ny2052jgaay6s1b62-cctools-binutils-darwin-
wrapper-16.0.6-973.0.1
/nix/store/jc5qxbblhm5cfyrra34j32hhaw15n040-cctools-llvm-16.0.6-973.0.1
/nix/store/q599hm4gzlrc4043667vzhanhbl10gr3-cctools-port-973.0.1
/nix/store/jai0wjrljly8l0z8vc72k38rldr684kvg-clang-16.0.6
/nix/store/wqbc85mi1f3njqnfb03s28dxi2ldfz2-clang-16.0.6-lib
/nix/store/cd7xf7gc37cvlvffa1124pv7bpxpnvw-clang-wrapper-16.0.6
/nix/store/ahzc3gy5pja4s1mbbgzvwz63r79kcs-compiler-rt-libc-16.0.6
/nix/store/frihdc7crah5h9506d9jf2qdnynkf4i-compiler-rt-libc-16.0.6-dev
/nix/store/f40kqz7zkz4xgzx3ai2nmm9rlw4bfff1-coreutils-9.5
/nix/store/dprl25x3xcrd18vkd65ha66nrhryn76-cowsay-3.7.0
/nix/store/nb9c4krs23rdiyyd0k2cs96ghwc4c7p-diffutils-3.10
/nix/store/kfjmw6x4zqvfaqq7883b1zcrgei77zg-ed-1.20.2
/nix/store/wqan875g7pwwlgrpray11z816z0cdrawy-expand-response-params
/nix/store/qw72mgrw5bfs96y1n5rbxp5ahfkz6rjb-file-5.45
/nix/store/ig9bnnl3bsrd9f3fybasf0hfahzfmjd-findutils-4.9.0
/nix/store/qpgn591wvsv242q36lh78k6d9l4g42c1-gawk-5.2.2
/nix/store/bb05ljb28ygzzxv9dlkja48xipjdj7r-gdbm-1.23
/nix/store/6x7jl3nggyi8kyrcqprhwp0m9wm49b-gemfile-and-lockfile
/nix/store/6kf67lpxagbi4vmn4mhvbg777yx19hb-gettext-0.21.1
/nix/store/3m26n63708n577wj6k94kvqa6bnjqb7-gmp-with-cxx-6.3.0
/nix/store/jc2fnn4ms15w41gfyrfw96lnlcb0hs8-gnugrep-3.11
/nix/store/18317yygjxf8mgpcmdznyiwmwingm0s3z-gnumake-4.4.1
/nix/store/mslj8gvr6lbniv4s6xc6dgkx47cirsgm-gnused-4.9
/nix/store/i3f22506jr66pxsqijammg5h1b574yxa-gnutar-1.35
/nix/store/vmyh2f5gnjxy32g3jrd2f2wysmr000nz-gzip-1.13
/nix/store/439jc4b81cwa7p238f4a1znwfr8bw9xd-libc++-16.0.6
/nix/store/yfdyzsf3d77vfaczwrjyrs9bapf9ph54-libc++-16.0.6-dev
/nix/store/rcb6561jzfkav59ba54jkrckrynmxll-libffi-3.4.6
/nix/store/ahx23nb68i341nm9x549zi8qqrxi88-libffi-3.4.6-dev
/nix/store/rwvj1ngkgl1lzl9myr6wdhx718l8y7b-libiconv-99
/nix/store/ml1yvhvab6da4lsq5w3h1vk314vhm98qq-libtapi-1100.0.11
/nix/store/0r3j9x74f4dfhpzadkccz2cagbf05rym-libunwind-35.3
/nix/store/nrxpvm5hkmnb3w7zcl2kbvsc8v1q2b8b-libxcrypt-4.4.36
/nix/store/r6c398pz6x4684f34n0wf6miczf497ga-libxml2-2.12.9
/nix/store/9ympnkyf8zxjlyxd2hv28v5xislhj3s-libyaml-0.2.5
/nix/store/v79r4jib27xzyyi2myxj0dbp68sbbjn0-libyaml-0.2.5-dev
/nix/store/ky7wqgvxv4gjjygc3ldp41qvwa11z6qv-lld-16.0.6
/nix/store/al33i5m53h04h0wa6r6clrmq6w2ran66-llvm-16.0.6
/nix/store/fb7y8z2wv2pcr55qc4kr5bd0xv31isws-llvm-16.0.6-lib
/nix/store/4lcwmxm1i20jhsmbldzadaiqp06n9z7-llvm-binutils-16.0.6
/nix/store/wl249kla8jy45a8nx2r51gm1fdm34qcg-ncurses-6.4
/nix/store/pzjnf8wy3263gjm1p1bp1k20p599pykz-ncurses-6.4-dev
/nix/store/45vf51gfcb133vicjc2ldljlnv7b1yn1-ncurses-6.4-man
/nix/store/7m6wljaqr0yfk470hlsqj1ylp6yw5j3c-objc4-709.1
/nix/store/mj0c9cjs4czfzh6grblrk2yj104snqj-openssl-3.0.14
/nix/store/4py3l8hm8w5fz1gs29j9h96p24h5cc26-openssl-3.0.14-bin
/nix/store/64ykg95y6dp7adz84qhwihbcvng4xk3-openssl-3.0.14-dev
/nix/store/mf5g79x7p0l8y5w0b90phh6kxji3nzpl-patch-2.7.6
/nix/store/59av6kaxhqlvy10gbd9vqbsmrn93z98k-pcre2-10.43
/nix/store/mpy28ybkwp74c4wafghqlx1n53frf3m-perl-5.38.2
/nix/store/2z76vdh0rhy9acryqd00kxy35vl085rf-readline-8.2p10
/nix/store/db1y15h93dd5r98ly6wrsck2i79z90xv-readline-8.2p10-dev
/nix/store/pl0bq355ck7s3c1d67nivla91bqxgmyp-ruby-3.1.6
/nix/store/2jyvl76lrmhbsfv1ha3xncff6b907x8r-ruby3.1-lolcat-100.0.1
/nix/store/g9zqbhwbfm6l0j42aj39zw8bb7bpina-ruby3.1-manpages-0.6.1
/nix/store/im25fjncmgjpy1qj0nd7j6a3cfgf5j6x-ruby3.1-optimist-3.0.1
/nix/store/yw2ircs3qp9a0bgp7rchwlp4327lxvg-ruby3.1-paint-2.2.0
/nix/store/qswbhp0ykh18rl41ha7kx4q75vl9r22y-sdkroot-10.12
/nix/store/3jwbhlh5j7k891qck5lcw94swb6h4l35-stdenv-darwin
/nix/store/a76lsxd0dixidj1z5qyv3yx9xfnqz3h-stdenv-darwin
/nix/store/kg7bgcikfw8yp0kiw0bma00dyasi75vd-xz-5.4.7
/nix/store/vjcl44q974wcdskxzipma4m2ghnxcj8c-xz-5.4.7-bin
/nix/store/cf3jj03a76gz9crnb5r7ywlgljkk25sa-zlib-1.3.1
/nix/store/85372rj8kj2vhbr4j58g2y4zk99qv742-zlib-1.3.1-dev
copying path '/nix/store/7ia0cb68r08jc0zv437a7k9jsm42gcyr-bash-5.2p32' from
'https://cache.nixos.org'...
copying path '/nix/store/jyplylm4rx3zfpw0nxvji5d991ax5gci-adv_cmds-119-
locale' from 'https://cache.nixos.org'...
copying path '/nix/store/9pzs18bap493anxxik2ngf8q9jag2sx-apple-framework-
CoreFoundation' from 'https://cache.nixos.org'...
copying path '/nix/store/18317yygjxf8mgpcmdznyiwmwingm0s3z-gnumake-4.4.1' from
'https://cache.nixos.org'...
copying path '/nix/store/mslj8gvr6lbniv4s6xc6dgkx47cirsgm-gnused-4.9' from
'https://cache.nixos.org'...
copying path '/nix/store/qswbhp0ykh18rl41ha7kx4q75vl9r22y-sdkroot-10.12' from
'https://cache.nixos.org'...
copying path '/nix/store/f04k2hmye11ub6wvncbq44f7ucddc
```

copying path '/nix/store/19wksmmlxclwyr0vvsnc0gg41/wcsuug-
Libsystem-1238.60.2' from '<https://cache.nixos.org>'...
copying path '/nix/store/n9cp4mmgya6pcglpdk58bzhwdrwq8618-bzip2-1.0.8' from
'<https://cache.nixos.org>'...
copying path '/nix/store/wqan875g7pwwlgpray11z816z0cdrawy-expand-response-
params' from '<https://cache.nixos.org>'...
copying path '/nix/store/439jc4b81cwa7p238f4a1znwfr8bw9xd-libcxx-16.0.6' from
'<https://cache.nixos.org>'...
copying path '/nix/store/rwvj1ngkg11zlh9myr6wdhx718l8y7b-libconv-99' from
'<https://cache.nixos.org>'...
copying path '/nix/store/nrxpvm5hkmb3w7zcl2kbvsc8v1q2b8b-libxcrypt-4.4.36'
from '<https://cache.nixos.org>'...
copying path '/nix/store/59av6kaxhqlvy10gbd9vqbsmrn93z98k-pcre2-10.43' from
'<https://cache.nixos.org>'...
copying path '/nix/store/kg7bgcikfw8yp0kiw0bma00dyasi75vd-xz-5.4.7' from
'<https://cache.nixos.org>'...
copying path '/nix/store/cf3jj03a76gz9crnb5r7ywlgljkk25sa-zlib-1.3.1' from
'<https://cache.nixos.org>'...
copying path '/nix/store/6x7jl3nggyi8kyrccqprhwzp0m9wm49b-gemfile-and-
lockfile' from '<https://cache.nixos.org>'...
copying path '/nix/store/bb05lj28ygzzxv9dlkja48xipjdjf7r-gdbm-1.23' from
'<https://cache.nixos.org>'...
copying path '/nix/store/rcb6561jzfakvs9ba54jkrckrynmlxll-libffi-3.4.6' from
'<https://cache.nixos.org>'...
copying path '/nix/store/0r3j9x74f4dfhpzadkccz2cagbf05rym-libunwind-35.3'
from '<https://cache.nixos.org>'...
copying path '/nix/store/9ympnkyf8zxjlyxd2hv28v5xsislhj3s-libyaml-0.2.5' from
'<https://cache.nixos.org>'...
copying path '/nix/store/qw72mgrw5bfs96y1n5rbxp5ahfkz6rjb-file-5.45' from
'<https://cache.nixos.org>'...
copying path '/nix/store/wl249kla8jy45a8nx2r51gm1fdm34qcg-ncurses-6.4' from
'<https://cache.nixos.org>'...
copying path '/nix/store/f3q95y31k4smw1qwcgvf0mpm4n9la7yn-bzip2-1.0.8-bin'
from '<https://cache.nixos.org>'...
copying path '/nix/store/45vf51gfc133vicj2ldljlnv7b1yn1-ncurses-6.4-man'
from '<https://cache.nixos.org>'...
copying path '/nix/store/7m6wljaqr0yfk470hlqsj1ylp6yw5j3c-objc4-709.1' from
'<https://cache.nixos.org>'...
copying path '/nix/store/mj0c9cjs4czfzh6grblrk2yjl04snqj-openssl-3.0.14'
from '<https://cache.nixos.org>'...
copying path '/nix/store/im25fjncmgjpy1qj0nd7j6a3cfgf5j6x-ruby3.1-
optimist-3.0.1' from '<https://cache.nixos.org>'...
copying path '/nix/store/yw2ircs3qp9a0bgp7rchwlp4327lxvg-ruby3.1-
paint-2.2.0' from '<https://cache.nixos.org>'...
copying path '/nix/store/kfjmpw6x4zqvfaqg7883b1zcrhci77zg-ed-1.20.2' from
'<https://cache.nixos.org>'...
copying path '/nix/store/ahzc3gy5pja4s1mbbgznvwz63r79kcs-gcc-rt-
libc-16.0.6' from '<https://cache.nixos.org>'...
copying path '/nix/store/3m26n63708n577wj6k94kvqa6bnjhqb7-gmp-with-cxx-6.3.0'
from '<https://cache.nixos.org>'...
copying path '/nix/store/vmyh2f5gnjxy32g3jrd2f2wysmr00nz-gzip-1.13' from
'<https://cache.nixos.org>'...
copying path '/nix/store/yfdyzsf3d77vfaczwrjyrs9bapf9ph54-libcxx-16.0.6-dev'
from '<https://cache.nixos.org>'...
copying path '/nix/store/ahx23nb68i341nm9x549zi8qrxlig88-libffi-3.4.6-dev'
from '<https://cache.nixos.org>'...
copying path '/nix/store/g9zqbhwbfm6l0j42aj39zw8bb7bpina-ruby3.1-
manpages-0.6.1' from '<https://cache.nixos.org>'...
copying path '/nix/store/v79r4jib27zyyi2myxj0dbp68sbbjn0-libyaml-0.2.5-dev'
from '<https://cache.nixos.org>'...
copying path '/nix/store/vjcl44q974wcdskxzipma4m2ghnxjc8c-xz-5.4.7-bin' from
'<https://cache.nixos.org>'...
copying path '/nix/store/mf5g79x7p0l8y5w0b90phh6kxji3nzpl-patch-2.7.6' from
'<https://cache.nixos.org>'...
copying path '/nix/store/85372rj8kj2vhbr4j58g2y4zk99qv742-zlib-1.3.1-dev'
from '<https://cache.nixos.org>'...
copying path '/nix/store/f40kqz7zkz4xgzx3ai2nm9rlw4bffl1-coreutils-9.5' from
'<https://cache.nixos.org>'...
copying path '/nix/store/frihdc7crah5h9506d9jf2qdnynkf4i-compiler-rt-
libc-16.0.6-dev' from '<https://cache.nixos.org>'...
copying path '/nix/store/4py3l8hm8w5fz1gs29j9h96p24h5cc26-openssl-3.0.14-bin'
from '<https://cache.nixos.org>'...
copying path '/nix/store/nb9c4krs23rdiyjyd0k2cs96ghwc4c7p-diffutils-3.10'
from '<https://cache.nixos.org>'...
copying path '/nix/store/ig9bnnl3bsrd9f3fybasf0hfazfmidj-findutils-4.9.0'
from '<https://cache.nixos.org>'...
copying path '/nix/store/6kfv67lxpagbi4vmn4mhvbg777yxl9hb-gettext-0.21.1'
from '<https://cache.nixos.org>'...
copying path '/nix/store/r6c398pz6x4684f34n0wf6miczf497ga-libxml2-2.12.9'
from '<https://cache.nixos.org>'...
copying path '/nix/store/jc2fnn4ms15w41gfyrfw96lnlccb0hs8-gnugrep-3.11' from
'<https://cache.nixos.org>'...
copying path '/nix/store/64yggk95y6dp7adz84qhwiyhbcvng4xk3-openssl-3.0.14-dev'
from '<https://cache.nixos.org>'...

```

copying path '/nix/store/mpy28yybkwp74c4wafghqlx1n53frf3m-perl-5.38.2' from
'https://cache.nixos.org'...
copying path '/nix/store/m1iyhvab6da4lsq5w3h1vk314vhm98qq-libtapi-1100.0.11'
from 'https://cache.nixos.org'...
copying path '/nix/store/fb7y8z2wv2pcr55qc4kr5bd0xv31isws-llvm-16.0.6-lib'
from 'https://cache.nixos.org'...
copying path '/nix/store/pzjnf8wy3263gjm1p1bp1k20p599pykz-ncurses-6.4-dev'
from 'https://cache.nixos.org'...
copying path '/nix/store/2z76vdh0rhy9acryqd00kxy35vl085rf-readline-8.2p10'
from 'https://cache.nixos.org'...
copying path '/nix/store/db1y15h93dd5r98ly6wrsck2i79z90xv-readline-8.2p10-
dev' from 'https://cache.nixos.org'...
copying path '/nix/store/pl0bq355ck7s3c1d67nivla91bqxgmyr-ruby-3.1.6' from
'https://cache.nixos.org'...
copying path '/nix/store/dqwq90k5arnpsj030avqjl8p8ikqyyjw-binutils-2.41-lib'
from 'https://cache.nixos.org'...
copying path '/nix/store/qpgn591wvsv242q36lh78k6d9l4g42c1-gawk-5.2.2' from
'https://cache.nixos.org'...
copying path '/nix/store/i3f22506jr66pxsqijammg5h1b574yxa-gnutar-1.35' from
'https://cache.nixos.org'...
copying path '/nix/store/3jwbhlh5j7k891qck5lcw94swb6h4l35-stdenv-darwin' from
'https://cache.nixos.org'...
copying path '/nix/store/lfb99d5radvax528l6aadyd5a8fa7lpm-binutils-2.41' from
'https://cache.nixos.org'...
building '/nix/store/zp91i2w23fvrzkxpjjbsp684h4r7rlr1-builder.pl.drv'...
copying path '/nix/store/q599hm4gzlrc4043667vzhanhbl10gr3-cctools-
port-973.0.1' from 'https://cache.nixos.org'...
copying path '/nix/store/dprl25x3xcrd18vkdn65ha66nrhryn76-cowsay-3.7.0' from
'https://cache.nixos.org'...
copying path '/nix/store/2y36i04pshmfykz4a6nd3jiihlngz4c8-bundler-2.5.9' from
'https://cache.nixos.org'...
copying path '/nix/store/2jyvl76lrmhbsfvlha3xncffb6907x8r-ruby3.1-
lolcat-100.0.1' from 'https://cache.nixos.org'...
building '/nix/store/zig0y28vz6y2dx7i5vh0pmrj7plgvamq-lolcat-100.0.1.drv'...
created 15 symlinks in user environment
building '/nix/store/n76nv3d8zl36q4vhkvxmh056k7q5kb4i-lolcat-100.0.1.drv'...
/nix/store/yypx17hx8q9yyqzmfl220mc02if56pmw-
lolcat-100.0.1/lib/ruby/gems/3.1.0/gems/lolcat-100.0.1/man/lolcat.6
copying path '/nix/store/wqbc85mlf3njqnfb03s28dxi2ldfz2-clang-16.0.6-lib'
from 'https://cache.nixos.org'...
copying path '/nix/store/ky7wqgvxv4gjjg3cldp41qvwqa11z6qv-lld-16.0.6' from
'https://cache.nixos.org'...
copying path '/nix/store/al33i5m53h04h0wa6r6clrmq6w2ran66-llvm-16.0.6' from
'https://cache.nixos.org'...
copying path '/nix/store/4lcwxmq1i20jhsmbldzadaiqp06n9z7-llvm-
binutils-16.0.6' from 'https://cache.nixos.org'...
copying path '/nix/store/jai0wjrjly8l0z8vc72k38rlldr684kvg-clang-16.0.6' from
'https://cache.nixos.org'...
copying path '/nix/store/jc5qxbblhm5cfyrra34j32hhaw15n040-cctools-
llvm-16.0.6-973.0.1' from 'https://cache.nixos.org'...
copying path '/nix/store/268cqlnw6ww917p8alrls1wjxydqyvwq-cctools-binutils-
darwin-16.0.6-973.0.1' from 'https://cache.nixos.org'...
copying path '/nix/store/rgiirnhx8b9ign0ny2052jgaay6s1b62-cctools-binutils-
darwin-wrapper-16.0.6-973.0.1' from 'https://cache.nixos.org'...
copying path '/nix/store/cd7xf7gc37cvlvffa1124pv7bxbpnvw-clang-
wrapper-16.0.6' from 'https://cache.nixos.org'...
copying path '/nix/store/a76lsxd0dixidxj1z5qyv3yx9xfnqz3h-stdenv-darwin' from
'https://cache.nixos.org'...

```

```
[nix-shell:~/nix-demo]$ cowsay "hi nix" | lolcat
```

```

< hi nix >
  _____
 /         \
(oo)_____)
(  )_____)
 ||-----w ||
 ||         ||

```

```
[nix-shell:~/nix-demo]$
```

Finally done!!!!!!